

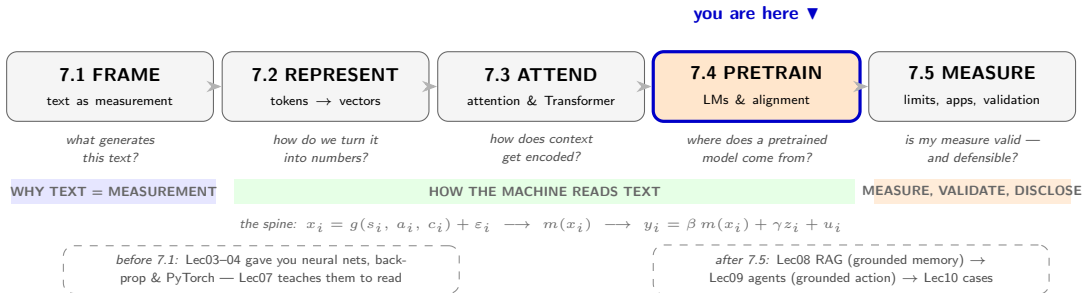
AI for Economic Research: Dynamic Models, Language, and Agents

Lecture 7.4: Pretrained Language Models and Alignment

Zhigang Feng

2026-07-07 05:00:23

Lecture 7 in One Map

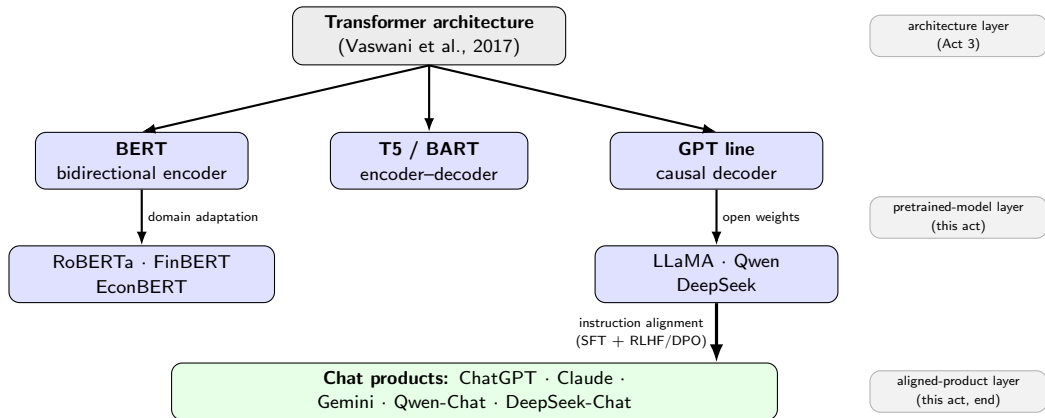


From Architecture to Model

Products vs. architectures: a three-layer family tree

Products vs. Architectures: The Family Tree

ChatGPT is not a model family — it is the aligned product of one. Three layers to keep straight:



Encoders (left branch) are the measurement workhorses — classification, embeddings, similarity. **Decoders** (right

branch) generate; alignment turns them into assistants. Calling ChatGPT means using all three layers at once: decoder

Pretraining: Self-Supervised Language Modeling

- **Data:** hundreds of billions to trillions of tokens of natural text and code (web pages, books, papers, code repositories, filings, news, ...).
- **Objectives:**
 - **Causal LM** (GPT family): $\mathcal{L}(\theta) = - \sum_s \sum_i \log P_\theta(t_s^{(i+1)} | t_s^{(1:i)})$. *In words:* at every position in every training sequence, the model outputs a vocabulary distribution; we penalize it by the negative log-probability of the token that *actually* appeared.
 - **Masked LM** (BERT family): hide a random subset of tokens and predict them bidirectionally, $\mathcal{L}(\theta) = - \sum_{i \in \text{masked}} \log P_\theta(t^{(i)} | t^{(\setminus i)})$.
- **This is maximum-likelihood estimation** of a conditional distribution. The novelty vs. classical statistics is the parameterization (10^{11} -parameter neural network) and the scale of the data — not the statistical objective.
- **Optimization:** Adam/AdamW, large batches, warmup + cosine LR, gradient clipping, mixed precision. **Compute:** weeks–months on thousands of accelerators, 10^{23} – 10^{25} FLOPs. No human labels — the next token *is* the label.

Two Pretraining Objectives: MLM vs. CLM

Fill in hidden words from both sides, or predict strictly left to right — the objective determines what the model is good for.

	BERT (masked LM)	GPT (causal LM)
Objective	mask ~15% of tokens; predict them from <i>both</i> sides	predict every next token from the left context
Context direction	bidirectional	causal (left-to-right)
Training signal	masked positions only	every position
Natural strength	understanding: disambiguation, classification, embeddings	generation: continuation, summarization, dialogue
Typical econ use	stance scoring, 10-K classification, similarity, retrieval	summaries, structured extraction, zero-shot labeling
Adaptation cost	low: linear head on [CLS]	prompt design or instruction tuning

Where bidirectionality pays: disambiguation needs the *right-hand* context too — *rate* (policy instrument vs. unemployment statistic), *liability* (accounting line vs. everyday negative). For pure measurement tasks on fixed text, the older BERT family is often still the better *instrument*: cheaper, 5/28

Generating the Next Token

- **After pretraining** we have a trained θ . Generation uses that θ to extend a prompt.
- Given a prompt $t^{(1)}, \dots, t^{(n)}$, run the Transformer to obtain $e'^{(n)}$.
- Compute logits and softmax:

$$P(t^{(n+1)} = w \mid t^{(1)}, \dots, t^{(n)}) = \text{softmax}(W_{\text{vocab}} e'^{(n)})_w.$$

- **Pick** a token according to a sampling strategy, **append** it to the context, and **repeat** until a stop token, a length limit, or an end-of-sequence marker.
- **Cost:** naively, generating n tokens is $O(n^2)$ in attention; practical implementations cache keys and values.

Tokens, Cost, and Reproducibility

Frontier-LLM economics (list prices checked 2026-07):

Model	In \$/M	Out \$/M
Claude Opus 4.8	5	25
GPT-5.4	2.50	15
Gemini 3.1 Pro	2	12
Qwen / DeepSeek / LLaMA (local)	GPU-hours, not API fees	

Frontier context windows now span $\sim 200\text{k}$ – 1M tokens. This is a *dated snapshot*: list prices, context limits, and the flagship generation move every quarter. **Lock the snapshot — model id, prices, and date — the day you run the study.**

What this means for measurement:

- Yin–Vu–Persico (T5a/T5b) run 18,797 O*NET tasks $\times$ 3 frontier annotators $\approx$ 56,000 calls. At $\sim 2\text{k}$ tokens/call $\Rightarrow \sim 110\text{M}$ tokens per provider. *Per-call* cost is trivial; *per-study* cost is not.
- **Reproducibility budget.** A pinned model version + frozen prompt + fixed seed (where supported) + recorded date is one “equipment item.” Treat it as you would a survey instrument.
- **Three things to archive** so a referee can re-run you:
 - the exact API call (model id, temperature, seed),
 - the inputs (hashes),
 - the raw outputs *before* any post-processing

Sampling Strategies

- **Greedy:** pick $\arg \max_w P(w \mid \cdot)$. Deterministic, but often repetitive and bland.
- **Top- k sampling:** restrict to the k most likely tokens, renormalize, then sample. Cuts off the long tail of low-probability nonsense.
- **Top- p (nucleus) sampling:** restrict to the smallest set of tokens whose cumulative probability exceeds p (e.g. 0.9). Adapts to how peaked the distribution is.
- **Temperature τ :** rescale logits by $1/\tau$ before softmax.
 - $\tau \rightarrow 0$: deterministic, like greedy
 - $\tau = 1$: model's natural distribution
 - $\tau > 1$: more diverse, more creative, more risky
- **For empirical research, set $\tau = 0$ (and report it).** Reproducibility is more valuable than creativity when measuring something. For generative tasks (drafting, synthetic-text experiments), $\tau \in [0.7, 1.0]$ with top- $p \in [0.9, 0.95]$ is the usual compromise — but record the parameters and seed even then.

Steering Without Training

Prompting, in-context learning, and schema-constrained output

Prompting: Zero-Shot, Few-Shot, In-Context Learning

Zero-shot. No examples in the prompt.

```
Classify this FOMC sentence as
hawkish, dovish, or neutral:
"Inflation remains elevated and
the Committee is highly atten-
tive."
Label:
```

→ model: hawkish. Works when the task name is in the pretraining distribution; brittle on rare or domain-specific tasks.

Few-shot. 2–8 exemplars in the prompt.

```
"Inflation is too high."
Label: hawkish
"Risks are tilted to the down-
side."
Label: dovish
"...highly attentive."
Label:
```

→ accuracy jumps without retraining — the model infers the label space and the decision rule from the demonstrations.

In-context learning (ICL).

- No gradient updates.
- Demonstrations act as a temporary “task vector” inside the forward pass.
- Empirically: accuracy scales with the number and quality of exemplars; ordering, label format, and class balance all matter.
- This is the workhorse pattern for LLM-as-rater measurement (T5a/T5b, Eloundou et al.).

In-Context Learning as Implicit Bayesian Inference

- **Xie, Raghunathan, Liang, Ma** (ICLR 2022; arXiv 2111.02080) frame ICL as Bayesian inference over a latent “task” variable τ :

$$P(y_{\text{target}} \mid x_{\text{target}}, \text{demos}) = \int P(y_{\text{target}} \mid x_{\text{target}}, \tau) \underbrace{P(\tau \mid \text{demos})}_{\text{posterior over task}} d\tau.$$

The prompt’s demonstrations *update* the model’s beliefs about *which* task to perform.

- **Three predictions, all observed empirically:**
 - More (relevant) demonstrations $\Rightarrow$ posterior $P(\tau \mid \text{demos})$ concentrates on the intended task $\Rightarrow$ accuracy rises.
 - Demonstration *ordering* and *label format* are not nuisances — they shape the posterior.
 - Out-of-distribution exemplars push the posterior toward the wrong τ ; ICL accuracy collapses.
- **Take-away.** *Your prompt is your prior.* The very mechanism that lets a frontier LLM adapt to a new economic measurement task without retraining is the same mechanism that produces sycophancy, prior-contamination, and template sensitivity. Lec09 returns to this as the *context dilemma* — the conditioning that helps also biases.

Structured Output: JSON as a Measurement Tool

Free-text answers are unstable. A rubric that asks for “hawkish / dovish / neutral” may produce Hawkish., ‘hawkish’, The label is hawkish., or worse. Post-processing such variation is brittle and undermines reproducibility.

Solution. Force the model to emit a fixed-schema JSON object. Every provider exposes a “structured output” / “tool use” API that constrains decoding (logit masking over disallowed tokens) so the output is *guaranteed* to parse.

Mechanism. The model is still next-token predicting. The provider’s API simply masks tokens that would violate the schema — the next-token distribution is restricted to syntactically valid continuations.

Why it bridges to Lec09. An “agent” is just an LLM whose schema names *tools to call*, not labels to output. The same machinery.

Sentiment-extraction schema:

```
{
  "stance": "hawkish|dovish|neutral",
  "confidence": 0.0,
  "evidence_phrase": ""
}
```

Tool-call schema (Lec09 preview):

```
{
  "tool": "fred_query",
  "args": {
    "series": "UNRATE",
    "start": "2020-01"
  }
}
```

For econ research: schemas *enforce* the rubric (no off-rubric labels), expose the model’s evidence (auditable), and let you compute Cohen’s κ on parseable output. Every data-extraction pipeline in T5b ships with one.

Fine-Tuning for Economics

From general representations to your domain, at single-GPU cost

Fine-Tuning for Economics

- A pretrained LLM has *general* linguistic knowledge. Economic tasks often benefit from a small, targeted update.
- **Three common strategies:**
 - **Full fine-tuning:** update every parameter. Highest quality, highest cost. Risks overwriting general knowledge with limited task data (catastrophic forgetting below $\sim 1,000$ labels).
 - **Adapters / LoRA** (Low-Rank Adaptation): freeze the original weight W and learn a low-rank update AB^T . Trains $\sim 0.1\%$ of the parameters at near full-FT quality. *Preferred* for most economics applications.
 - **Domain Adaptive Pretraining (DAPT):** continue pretraining on a domain corpus (FOMC statements, MD&A texts, central bank speeches) before fine-tuning on the labeled task.
- **Examples:** sentiment classification on FOMC statements, 10-K segment labeling, NER for corporate filings, summarization of monetary policy reports.

Corpus choice is a validity decision, not a tuning trick. Scoring “financing pressure” in Chinese MD&A with generic BERT-Chinese vs. after DAPT on five years of filings is not just an accuracy difference: the generic model’s confusion over disclosure jargon (“liquidity management”, “refinancing risk”) becomes measurement error in $m(x_i)$ attenuating $\hat{\beta}$ exactly like a badly chosen proxy^{12/28}

Why LoRA Works: The Low-Rank Intuition

- **The empirical fact** (Hu et al. 2021, Fig. 3): the task-specific weight update $\Delta W = W_{\text{fine-tuned}} - W_{\text{pretrained}}$ is *nearly low-rank*. Most of the singular-value mass lives in ≤ 10 directions.
- **Economic analogy.** The cross-sectional variation in FOMC sentence stance is not 7B-dimensional. It lives on a small number of latent dimensions: hawk–dove, topic (inflation vs. employment), tense (description vs. forecast), and a handful of speaker idioms. *The fine-tuning task lives in a low-rank subspace; the parameter update should too.*
- **When rank r matters:**
 - *Rank 4–8 suffices* for stance/sentiment classification, NER, paraphrasing — tasks that re-weight existing knowledge.
 - *Rank 16–64* for instruction following, multi-turn dialogue — tasks that compose existing skills in new ways.
 - *LoRA fails* when the task demands broad lexical change (e.g. adapting a generic LLM to medieval Latin) or memorizing new factual associations — those need full DAPT or RAG, not a low-rank patch.
- **Why this matters for empirical economists.** The compute cost of fine-tuning a 70B model on

Fine-Tuning: LoRA Mechanics

- **Idea:** instead of updating every weight matrix W in the model, freeze W and learn a *low-rank correction*:

$$W_{\text{eff}} = W + AB^{\top}, \quad A \in \mathbb{R}^{d \times r}, \quad B \in \mathbb{R}^{k \times r}, \quad r \ll \min(d, k).$$

Only A and B are trainable during fine-tuning; the original W stays fixed.

- **Parameter savings** (LLaMA-7B):
 - Full fine-tuning updates $\sim 7 \times 10^9$ parameters.
 - LoRA with $r = 8$ on attention projections updates $\sim 4 \times 10^6$ parameters ($\approx 0.06\%$ of the model).
 - Near-equivalent task quality for most classification, sentiment, and style tasks.
- **Why it matters for economists:** a 7B or even 70B base model can be adapted on a single GPU or a modest cloud instance — no industrial-scale hardware required.
- **Inference:** after training, merge the update back into the weights ($W \leftarrow W + AB^{\top}$); deployment cost is identical to the base model. The adapter itself is tens of MB — swap it per task, version it, share it.
- **Tooling:** HuggingFace `peft` wraps any pretrained model with LoRA adapters in a few lines of code.

Fine-Tuning: Workflow for an Economics Task

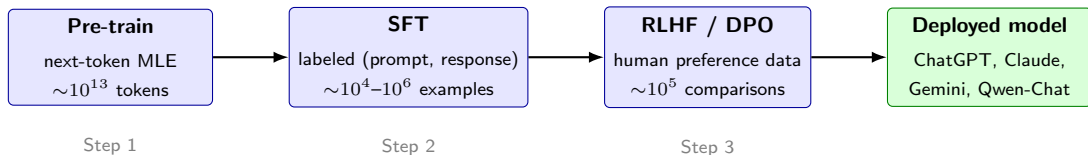
Running example: classify FOMC statements as hawkish / dovish / neutral.

- **Dataset:** 500–2,000 hand-coded statements (e.g. Hansen–Kazinnik, 2023). Split by *meeting date* — train on 1994–2015, validate on 2016–2019, test on 2020–2024 — to avoid leaking future regimes into training.
- **Base model:** an encoder (BERT / RoBERTa) with a classification head is usually enough and cheaper than decoder fine-tuning for three-way classification.
- **Task formatting:** prepend [CLS], tokenize and pad to a fixed length, attach a softmax head over 3 classes to the [CLS] output vector.
- **Hyperparameters (LoRA defaults):** learning rate $1\text{--}3 \times 10^{-4}$, rank $r = 8$, batch size 16–32, 3–5 epochs, early stopping on validation macro-F1.
- **Evaluation:** report macro-F1 and a per-class confusion matrix; hand-audit 20–30 misclassifications — most errors reveal label noise or systematic biases worth fixing before scaling.
- **Reference template:** CentralBankRoBERTa (T5b) follows exactly this recipe, with a DAPT pass on the FOMC corpus preceding the labeled fine-tuning.

Alignment

From “completes text” to “answers questions” — and what that does to measurement

From Base Model to Assistant: Three Stages



- **Pre-training** alone gives a model that completes text. It does *not* answer questions or follow instructions reliably — ask a base model a question and its most likely continuation is *another phrasing of the question*.
- **SFT + RLHF** is what turns a base model into a ChatGPT-style assistant.
- Each stage uses progressively less data but more targeted human effort; step 1 consumes $>90\%$ of the compute, steps 2–3 determine *how the model answers*.

Supervised Fine-Tuning (SFT) and Instruction Tuning

- **Setup:** hand-curated dataset of (instruction, ideal response) pairs — often 10,000–1,000,000 examples.
- **Loss:** same next-token cross-entropy as pre-training, but only on the response tokens, conditional on the instruction.

$$\mathcal{L}_{\text{SFT}}(\theta) = - \sum_{(x,y)} \sum_{t=1}^{|y|} \log P_{\theta}(y_t \mid x, y_{<t}).$$

- **What this does:**
 - Teaches format: “when asked a question, return an answer, not a continuation.”
 - Teaches refusal patterns and helpfulness conventions.
 - Often combined with diverse instruction collections (FLAN, Alpaca, ShareGPT).
- **What it cannot do:** each instruction carries *one* demonstration answer, so the model never learns which of two decent answers is *better* — that gap is what preference learning (next slide) fills.
- **Connection to economics fine-tuning above:** full FT and LoRA from earlier slides are also used here — SFT is just fine-tuning with a particular kind of data (instruction-response pairs)

RLHF: Aligning to Human Preferences

- **Reinforcement Learning from Human Feedback** (Christiano et al., 2017; Ouyang et al., 2022):

1. For each prompt, sample two responses y_A, y_B from the SFT model.
2. A human labels which is better: $y_A \succ y_B$.
3. Train a **reward model** $r_\phi(x, y)$ on these comparisons (Bradley–Terry):

$$P(y_A \succ y_B \mid x) = \sigma(r_\phi(x, y_A) - r_\phi(x, y_B)).$$

4. Fine-tune the LLM with **PPO** to maximize $\mathbb{E}[r_\phi(x, y)]$ subject to a KL penalty against the SFT policy:

$$\max_{\theta} \mathbb{E}_{x, y \sim \pi_{\theta}} [r_\phi(x, y)] - \beta \text{KL}(\pi_{\theta} \parallel \pi_{\text{SFT}}).$$

- **The KL penalty is essential:** without it, the policy over-optimizes the reward model and “hacks” it (Goodhart’s law in action — repetitive phrasing, over-apologizing).
- **DPO** (Rafailov et al. 2023) skips the reward model entirely: a supervised pairwise loss on the same preference data — no PPO rollouts, much cheaper, numerically stable. Most open-weights chat models (LLaMA-Instruct, Qwen-Chat, Mistral-Instruct) now ship with DPO or a variant (IPO, KTO, ORPO). The algorithm moves fast; the goal does not.

Why Alignment Matters: Helpful, Honest, Harmless

- Anthropic's "HHH" framing (Askell et al. 2021) is the standard rubric:
 - **Helpful**: actually solves the user's task.
 - **Honest**: does not assert things it has no basis for; expresses calibrated uncertainty.
 - **Harmless**: refuses to produce content that materially harms others.
- **Tensions are real**:
 - Helpfulness can pull against harmlessness (over-refusal vs. over-compliance).
 - Honesty can pull against helpfulness ("I don't know" is honest but may be unhelpful).
 - Alignment is a multi-objective trade-off, not a scalar.
- **For research use**:
 - Aligned models are easier to use for first-pass tasks.
 - But alignment can suppress edge-case completions you might want (e.g. negative sentiment templates). Sometimes a base model is preferable for measurement.
 - Constitutional AI (Bai et al. 2022): replace human comparisons with AI-generated critiques against a written constitution — scales the alignment budget.

Alignment's Measurement Consequence: The Version Is the Instrument

What preference tuning adds, preference tuning can silently change.

- SFT demonstrations and preference data are *proprietary, rolling* assets. From outside, all you observe is a version string: gpt-4-0613 vs. gpt-4-2024-08-06.
- Two versions can score the same FOMC sentence differently — not “smarter” or “dumber”, just a different internal prior about what a good answer looks like.
- Evidence: GPT-4's zero-shot hawk-dove labels agree with expert coders *better than fine-tuned encoders do* — and shift with prompt wording and API version (Hansen & Kazinnik 2023). The capability and the instability arrive together.
- Minimum disclosure whenever an aligned LLM produces a variable: provider, model name, exact version, call date, sampling parameters — and archive the raw outputs. (T5b turns this into a full workflow.)

An aligned LLM used for scoring is a **time-varying measurement instrument**. Version-pin it like lab equipment, and treat a version change as an instrument change — because it is one.

Scale

Power laws, compute-optimal training, and the emergence debate

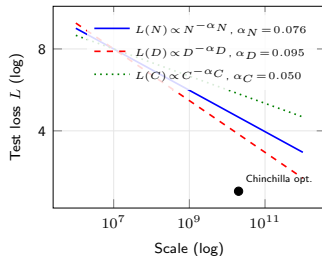
Kaplan Scaling Laws (2020): Loss is a Power Law

- Kaplan et al. (2020) trained dozens of GPT-style models varying N (params), D (data), C (compute).
- **Test loss falls as a power law in each:**

$$L(N) \approx \left(\frac{N_c}{N}\right)^{\alpha_N}, \quad L(D) \approx \left(\frac{D_c}{D}\right)^{\alpha_D},$$

with $\alpha_N \approx 0.076$, $\alpha_D \approx 0.095$.

- **Why this is striking:** no plateau over 7 orders of magnitude. Loss keeps falling smoothly with scale.
- **Implication:** for fixed compute $C \sim 6ND$, you can predict optimal (N, D) allocation *before* training.
- **For an economist:** a production function for capability — parameters and data as



Three power-laws on one set of axes: parameters N , data D , compute C . Chinchilla optimum is where the N -curve and the D -curve cross under a fixed compute budget.

Chinchilla (Hoffmann et al. 2022): $\text{Data} \propto \text{Params}$

- Kaplan suggested “mostly scale parameters.” Chinchilla revisited the question with a different fitting method and found:

$$\text{compute-optimal} \implies D^* \approx 20 \cdot N^*.$$

For each parameter, you want ~ 20 training tokens.

- **Earlier models were undertrained.** GPT-3 (175B params, 300B tokens) used ~ 1.7 tokens/param. A Chinchilla-optimal model at 175B would need $\sim 3.5\text{T}$ tokens.
- **Chinchilla itself:** 70B params trained on 1.4T tokens — beat 280B Gopher on most benchmarks.
- **Modern overtraining:** for inference cost reasons, frontier models now go *past* Chinchilla-optimal — e.g. Llama-3 8B trained on 15T tokens ($\sim 1,900$ tokens/param). Smaller, more-overtrained model = cheaper to deploy.
- **The economic frame:** pre-train compute is a one-time fixed cost; inference compute is a recurring marginal cost. Optimal allocation depends on expected serving volume — a marginal-vs-fixed-cost substitution any economist will recognize.

Emergent Capabilities — and the “Mirage” Critique

- **Wei et al. (2022):** on certain tasks (multi-step arithmetic, BIG-Bench, in-context learning), accuracy is near random for small models and then *jumps* sharply at a critical scale.
- Examples cited: 3-digit addition, word-in-context disambiguation, instruction-following.
- Coined the term *emergent capabilities*: not present in smaller siblings, suddenly present in larger ones.
- **Schaeffer, Miranda & Koyejo (2023, “Are Emergent Abilities a Mirage?”):**
 - The sharp jumps may be artifacts of *discontinuous metrics* (exact-match accuracy).
 - Switch to a continuous metric (e.g. token edit distance, partial credit) and the curves become smooth power-laws.
 - “Emergence” may be in our metrics, not in the underlying loss.
- **Where this leaves us:**
 - Capability *does* grow with scale — empirically robust.
 - Whether it grows “smoothly” or “in jumps” depends on how you measure.
 - For an economist: never infer “the model can do my annotation task” from a parameter count — test it *on that task*, against a gold standard.

The Open-Weights Menu — and Where It Runs

For confidential text, the model comes to the data — not the reverse.

Model	Type	L	d	Params	Pretraining corpus
BERT-Base	encoder	12	768	110M	books + Wikipedia (16GB)
RoBERTa	encoder	24	1024	355M	160GB multi-source
BERT-wwm-ext (HIT)	encoder	12	768	110M	Chinese wiki + Baidu Baike + news
GPT-3	decoder	96	12288	175B	filtered web (570GB)
Qwen-7B	decoder	32	4096	7B	Chinese-heavy, ~3T tokens
DeepSeek-V3	decoder (MoE)	61	7168	671B (37B active)	~14.8T tokens, zh+en

- **Compliance driver (China):** the Personal Information Protection Law (PIPL, 2021) and Data Security Law regulate cross-border transfer of personal data — sending firm filings with personal information or survey text to an offshore API is a legal exposure. Locally deployed Qwen / DeepSeek / ChatGLM / LLaMA is the audit-safe default.
- Long-context variants (128k tokens) now cover complete annual reports; MoE models (DeepSeek-V3: 671B total, 37B active per token) infer at mid-size dense cost.
- Same logic elsewhere: IRB-restricted transcripts, proprietary bank data, unpublished co-author drafts — local open weights or explicit data-handling sign-off.

Synthesis

Old friends, new engine — and the one equation, revisited

Transformer vs. LDA Topic Models

LDA / topic models

- Document = mixture of latent topics
- Bag of words: ignores order
- Output: topic-word and document-topic distributions
- Strength: interpretable themes
- Weakness: cannot generate coherent prose

Transformers

- Token-by-token, order-aware
- Captures syntax and long-range dependencies
- Output: contextual embeddings + a generative model
- Strength: rich generation, fluent prose, classification
- Weakness: less directly interpretable

When to use what: LDA is still excellent for thematic structure and quick descriptive analysis on small corpora (it powered Hansen–McMahon–Prat, T1). Transformers dominate when you need generation, classification, or context-aware embeddings.

Three Ways to Adapt an LLM

Method	What changes	Best for
Prompt engineering	Only the input text	Quick experiments, zero-cost adaptation
Fine-tuning (LoRA / DAPT)	Some model weights	New behavior or style on a fixed domain
Retrieval-Augmented Generation (RAG)	Relevant external context injected at inference time	New <i>facts</i> the model never saw

- All three are non-exclusive. In practice you often combine them.
- RAG addresses the limit we meet head-on in Act 5: *the model's knowledge has a cutoff date*.
- Lecture 8 is dedicated to RAG.

Revisiting: Why One Equation Enables So Much

- Now that we understand pretraining, fine-tuning, and generation, T1's opening claim makes mechanical sense:

$$p(t^{(n+1)} \mid t^{(1)}, \dots, t^{(n)}) \quad \text{is all there is.}$$

- **Summarizing a 10-K filing:** the prompt carries the document and the instruction; the model predicts summary tokens one by one because pretraining exposed it to millions of (long-document, summary) pairs.
- **Extracting FOMC stance:** fine-tuning on labeled central bank text shifts the next-token distribution so that stance-label tokens are most probable after a monetary-policy sentence.
- **Generating scenario forecasts:** condition on macro variables and a scenario description; next-token prediction fills in the narrative consistently with the conditioning.
- **Writing and debugging code:** the training corpus included billions of code tokens; the objective is identical — next token, given context.
- **The unifying insight:** all tasks are the same computation. Pretraining provides the knowledge; prompts and fine-tuning steer *which* next-token distribution we sample from. Every capability is a special case of the one equation.

Bridge: A Capable Model Is Still a Fallible Instrument

Everything an LLM does is the one equation — including its failures.

- The same next-token engine that summarizes a 10-K also *hallucinates* plausible fictions, drifts across API versions, and loses facts buried mid-context.
- **Act 5 turns the lens around:** first the five structural limits and what each does to research (T5a); then what embeddings and validated LLM calls can already measure — and the workflow that makes the measure defensible (T5b).

Arc: T1 FRAME → T2 REPRESENT → T3 ATTEND → T4 PRETRAIN → T5 MEASURE.